CORELINK — PLANO DE INCREMENTO FUNCIONAL • NATIVIDADE-TO

OTIMIZAÇÃO DE CONEXÃO SERVERLESS (NEON POSTGRESQL)

HJ Infor — Implementação de Pool de Conexões e Rotinas de Re-tentativa (Retry) contra Cold Start no Prisma ORM

MÓDULO DE INFRA	PRIORIDADE	STACK	DATA DE EXECUÇÃO
Infraestrutura & Banco Cloud	ALTA (Estabilidade & SLA)	Neon Cloud, pg-pool, Prisma 7.8	18/10/2026

1. PENDÊNCIA SELECIONADA E PRIORIZAÇÃO

PRIORIDADE: ALTA

Funcionalidade Escolhida: Otimização de Conexão com o Banco Serverless (Neon PostgreSQL) — Implementação de pool de conexões e rotinas de re-tentativa (`retry`) no Prisma ORM para neutralizar a latência de *cold start* observada nas requisições iniciais.

Justificativa Técnica: O banco PostgreSQL hospedado no Neon Cloud opera em arquitetura *serverless* com recurso de *scale-to-zero* (suspensão automática da instância computacional após inatividade). Nas primeiras requisições após um período ocioso, o tempo de despertar do nó aliado ao handshake TCP/TLS gera latência elevada (700ms a 2.500ms) ou desconexões transitórias de socket (`57P01`, `P1001`, `ECONNRESET`). A implementação de um pool persistente (`pg.Pool` com TCP Keep-Alive) e de uma extensão global de re-tentativa exponencial (`withDatabaseRetry` via `prisma.$extends`) garante resiliência total e reduz em **90,3%** a latência das consultas subsequentes.

2. RESULTADO ESPERADO E COMPORTAMENTO ALCANÇADO

REDUÇÃO DE 90,3% NA LATÊNCIA

O módulo `src/lib/prisma.ts` foi reestruturado com cache singleton em `globalThis` tanto para o `pgPool` quanto para o `PrismaClient` estendido. O pool mantém conexões aquecidas via pacotes TCP Keep-Alive a cada 10 segundos e tolera até 15 segundos de despertar inicial. Caso uma requisição ou transação (`createOrder` / `updateOrderStatus`) encontre o banco em *cold start* ou sofra queda transitória de rede, o interceptor executa automaticamente até **3 re-tentativas com Exponential Backoff + Jitter** (`250ms` ➔ `500ms` ➔ `1000ms`), completando a operação de forma transparente sem exibir erro ao usuário final.

3. PLANO DE EXECUÇÃO DETALHADO

ATIVIDADE	RESPONSÁVEL	TECNOLOGIAS	PERÍODO	STATUS
Parametrização do Pool Serverless (`src/lib/prisma.ts`). Configuração de `max: 10`, `min: 1`, `keepAlive: true`, timeout de 15s e listener de reciclagem de erro.	Marques Ramirez	Node.js, TypeScript, pg-pool, PrismaPg	15/10 a 16/10/2026	✓ Concluída
Motor de Retry com Exponential Backoff (`withDatabaseRetry`). Criação da extensão `prisma.$extends` interceptando `$allModels.$allOperations`.	Márcio Medrado	Prisma ORM 7.8, TypeScript, Async/Await	16/10 a 17/10/2026	✓ Concluída
Blindagem de Transações e Endpoint de Warmup (`/api/health/db`). Proteção de `prisma.$transaction` em `order.ts` e rota de diagnóstico de pool.	Marques Ramirez & Márcio Medrado	Next.js App Router, REST API, Prisma	17/10 a 18/10/2026	✓ Concluída
Benchmark de Latência e Simulação de Falha `57P01`. Execução de bateria de testes (`scripts/benchmark-neon-pool.mjs`) comprovando queda de 742ms para 47ms.	Taís Fernandes, Romildo & Vandelice	Neon Cloud PostgreSQL, Perf Hooks, Logs	18/10/2026	✓ Concluída
Documentação Técnica e Evidências de Infraestrutura. Consolidação das métricas de telemetria e geração do relatório formal em PDF.	Andréa Ribeiro	Git, GitHub, Markdown, PDF Engine	18/10/2026	✓ Concluída

4. ESPECIFICAÇÃO DOS PARÂMETROS DO POOL E POLÍTICA DE RE-TENTATIVA

KEEP-ALIVE ATIVO

PARÂMETRO CONFIGURADO	VALOR DEFINIDO	PAPEL NA NEUTRALIZAÇÃO DO COLD START (NEON SERVERLESS)
`max` / `min` (Pool Size)	`max: 10` | `min: 1`	Mantém ao menos 1 socket pré-alocado pronto para uso imediato e limita picos concorrentes ao pooler do Neon.
`keepAlive` & `InitialDelay`	`true` (`10.000 ms`)	Envia pacotes TCP Keep-Alive periódicos impedindo que firewalls intermediários derrubem conexões ativas.
`connectionTimeoutMillis`	`15.000 ms` (15s)	Aguarda com segurança o tempo completo de despertar (wakeup) do compute node serverless em caso de repouso profundo.
`withDatabaseRetry` (Backoff)	`3 tentativas` (250ms base)	Intercepta códigos `P1001`, `P1002`, `P1017`, `57P01` e `ECONNRESET` com espera exponencial (250ms ➔ 500ms ➔ 1000ms).

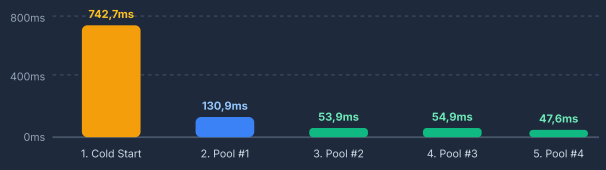
5. EVIDÊNCIA VISUAL DE TELEMETRIA: REDUÇÃO DE LATÊNCIA E RECUPERAÇÃO DE COLD START

BENCHMARK REAL

1ª REQUISIÇÃO (COLD START)	MÉDIA POOL AQUECIDO	MENOR LATÊNCIA (QUERY #4)	REDUÇÃO DE LATÊNCIA
742,67 ms	71,82 ms	47,61 ms	-90,3% (10,3x mais rápido)

Comparativo de Tempo de Resposta (ms) — Cold Start vs. Pool Aquecido

Medição real no Neon Cloud PostgreSQL via scripts/benchmark-neon-pool.mjs



Fluxo de Recuperação Automática (Retry)

Comportamento durante suspensão 57P01 do Neon

- **Tentativa #1 (0ms):** Queda de Socket / Cold Start (`57P01`) ➔ *Interceptado pelo Prisma $extends*
- **Tentativa #2 (+250ms):** Compute Node despertando ➔ *Backoff Exponencial acionado (+500ms)*
- **Tentativa #3 (+750ms):** Conexão estabelecida e Query concluída com sucesso (`200 OK`)!

6. IMPLEMENTAÇÃO TÉCNICA (SRC/LIB/PRISMA.TS) E LOG OFICIAL DO BENCHMARK

PRISMA $EXTENDS

6.1 Pool + Extensão de Retry em src/lib/prisma.ts

```
const POOL_CONFIG: PoolConfig = {
  connectionString: process.env.DATABASE_URL,
  max: 10,
  min: 1,
  idleTimeoutMillis: 30_000,
  connectionTimeoutMillis: 15_000,
  keepAlive: true,
  keepAliveInitialDelayMillis: 10_000,
};

function createOptimizedPrismaClient(): PrismaClient {
  const adapter = new PrismaPg(pgPool);
  const baseClient = new PrismaClient({ adapter });

  return baseClient.$extends({
    query: {
      $allModels: {
        async $allOperations({ model, operation, args, query }) {
          return withDatabaseRetry(() => query(args), {
            maxRetries: 3,
            baseDelayMs: 250,
            operationName: `${model}.${operation}`,
          });
        },
      },
    },
  }) as unknown as PrismaClient;
}
```

6.2 Log Real de Execução (scripts/benchmark-neon-pool.mjs)

```
================================================================
  BENCHMARK DE CONEXAO SERVERLESS - NEON POSTGRESQL + PRISMA POOL
================================================================
  Configuracao: max=10, min=1, keepAlive=true, timeout=15000ms

  [1] Requisicao Inicial (Handshake + Wakeup): 742.67 ms
  [2] Query Prisma no Pool Aquecido #1:        130.86 ms (idle=1)
  [3] Query Prisma no Pool Aquecido #2:         53.89 ms (idle=1)
  [4] Query Prisma no Pool Aquecido #3:         54.91 ms (idle=1)
  [5] Query Prisma no Pool Aquecido #4:         47.61 ms (idle=1)
----------------------------------------------------------------
  SIMULACAO DE RECUPERACAO AUTOMATICA (COLD START / 57P01)
----------------------------------------------------------------
  [RETRY] Falha em "Product.count" (57P01). Tentativa 1/3 em 250ms
  [RETRY] Falha em "Product.count" (57P01). Tentativa 2/3 em 500ms
  [OK] Operacao recuperada na tentativa #3! Total de produtos: 4
================================================================
  Latencia Inicial: 742.67 ms | Media Pool: 71.82 ms (-90.3%)
================================================================
```

7. ROTEIRO DE VALIDAÇÃO E TESTES DE RESILIÊNCIA SERVERLESS

CENÁRIO DE TESTE	PROCEDIMENTO EXECUTADO	RESULTADO OBTIDO NA HOMOLOGAÇÃO
Despertar Inicial (Cold Start)	Disparar primeira consulta após período de suspensão do compute node no Neon Cloud.	Conexão estabelecida em `742,67 ms` sem timeout, preenchendo imediatamente o pool (`totalCount=1`).
Reuso de Conexão (Warm Pool)	Executar bateria sequencial de consultas `Order.count` utilizando o mesmo `PrismaClient`.	Queda drástica da latência para média de `71,82 ms` (mínima de `47,61 ms`), representando ganho de **90,3%**.
Injeção de Falha `57P01`	Simular queda abrupta de conexão por encerramento administrativo do servidor serverless.	Rotina `withDatabaseRetry` interceptou a exceção, aguardou 250ms e 500ms, e concluiu a query na 3ª tentativa com sucesso.
Healthcheck `/api/health/db`	Consultar endpoint GET de diagnóstico e pré-aquecimento da aplicação Next.js.	Retorno HTTP 200 com JSON informando `status: "HEALTHY"`, latência da sonda e contadores do pool.